

abstract

Nouă exerciții pe un singur cuvânt-cheie — peste ierarhiile tale de figuri și de orar · Java · Adrian

Unde lucrezi: repo-ul `java-abstractizare`. Copiază acolo, așa cum sunt, pachetele tale din proiectul de moștenire: figurile geometrice în `app.figuri` și intervalele de orar în `app.orar`. Bug-urile semnalate în review se rezolvă **în repo-ul vechi** — aici ne interesează un singur lucru.

Nou, ai voie: `abstract` pe clasă și pe metodă, constructor `protected`.

Încă NU folosim: `interface`, `static`, `enum`, `record`, generice proprii, fișiere, `try/catch`, stream-uri. Vin la rând, separat.

Regula fișei: fiecare exercițiu are un rând **Gata când**. Se verifică prin **rulare** și prin ce scrie compilatorul, nu prin citirea codului. **Mesajele de eroare se notează pe fișă, cuvânt cu cuvânt** — jumătate din exerciții îți cer să produci intenționat o eroare.

Partea 1 — ce interzice `abstract`

nivel 1

1. Obiectul care n-ar trebui să existe

În `App`, rulează exact asta:

```
Desen d = new Desen(new ArrayList<>(List.of(
    new Cerc(new Punct(2, 3), 5),
    new Figura()
)));
d.afisare();

Orar orar = new Orar();
orar.adaugaInterval(new OraCurs(8, 50, "Mate", "A12"));
orar.adaugaInterval(new Interval(9, 10));
orar.afisare();
```

Notează **exact** ce apare pe rândul lui `new Figura()` și pe rândul lui `new Interval(9, 10)`.

Apoi scrie `abstract` în fața lui `class Figura` și a lui `class Interval` și compilează aceleași linii. Notează mesajul și numărul liniei.

Gata când: ai amândouă notate una sub alta — ieșirea de dinainte și eroarea de compilare de după.

Înainte de `abstract`, programul rula fără să se plângă. Ce anume „mergea” acolo — și cui îi era de folos rândul acela?

nivel 1

2. Metoda pe care o poți uita

Adaugă în `app.orar` un tip nou: **Examen** — ora de început, durata în minute, materia, sala și nota maximă. Scrie-l pe tiparul lui `OraCurs`, dar **nu scrie în el** `getTipInterval()`. Pune un examen în orar și rulează. Notează rândul afișat.

Apoi, în `Interval`, transformă `getTipInterval()` în metodă abstractă — declarată, fără corp — și compilează din nou.

Gata când: ai notat rândul din prima variantă și eroarea de compilare din a doua, împreună cu numele clasei care apare în ea.

În prima variantă nimeni nu s-a plâns. Cine ar fi trebuit să observe că orarul are un interval fără tip — tu, compilatorul, sau cel care citește orarul afișat?

3. Trei combinații care n-au sens

mai greu

Scrie-le pe rând, compilează după fiecare, notează mesajul, apoi revino:

```
final abstract class Interval { ... }

private abstract String getTipInterval();

abstract String getTipInterval() { return ""; }
```

Pentru fiecare, scrie un rând de explicație: ce promite `abstract` și cu ce se bate cap în cap celălalt cuvânt.

Gata când: ai trei mesaje notate și trei explicații scrise de tine — nu copiate din mesaj.

`abstract` spune „cineva de mai jos trebuie să termine treaba asta”. Care dintre cele trei linii face imposibil să existe acel „cineva de mai jos”?

Partea 2 — ce lasă în urmă `abstract`

4. Constructorul unei clase pe care n-o poți instanția

nivel 1

`Interval` e acum abstract, dar are în continuare un constructor. Pune în el un `System.out.println("constructor Interval")` și creează din `App` un singur `OraCurs`. Rulează. Apoi fă constructorul lui `Interval` `protected` și verifică din nou că `App` compilează.

Gata când: ai ieșirea rulării notată și poți răspunde în scris: dacă `new Interval(...)` e interzis, cine a chemat constructorul ăla și de câte ori?

Un `OraCurs` nu e „lângă” un `Interval`, ci **conține** partea de `Interval`. Cine o inițializează pe aceea, dacă nu constructorul ei?

5. Ce mai poți scrie cu un tip abstract

nivel 1

Scrie toate patru liniile, una câte una, și notează care compilează și care nu:

```
Interval i = new OraCurs(8, 50, "Mate", "A12");
List<Interval> lista = new ArrayList<>();
Interval[] vector = new Interval[3];
Interval x = new Interval(8, 50);
```

Gata când: ai lângă fiecare linie „compilează” sau mesajul de eroare, plus o frază care explică de ce linia 3 trece, deși pare că și ea creează `Interval`-uri.

Trei dintre linii vorbesc despre **unde ții** un obiect. Una singură vorbește despre **a-l fabrica**. Pe care o interzice `abstract`?

6. Metoda scrisă în clasa abstractă, care cheamă una nescrisă

mai greu

Acum `afisare()` există în `Interval` și e rescrisă în `OraCurs`, `Pauza`, `ActivitateLibera` și `Examen` — fiecare cheamă `super.afisare()` și mai adaugă ceva.

Rescrie `Interval` astfel încât **formatul rândului să existe într-un singur loc**, iar subclasele să nu mai conțină deloc metoda `afisare`. Tot ce diferă între tipuri rămâne **declarat abstract** în `Interval`.

Gata când: `grep -c afisare` pe fiecare subclasă dă `0`; ieșirea e identică rând cu rând cu cea de dinainte; iar dacă mai adaugi încă un tip de interval, rândul lui apare corect **fără să modifice `Interval`**.

O clasă abstractă are voie și cu metode gata scrise, și cu metode doar declarate. Din rândul afișat, ce bucată e aceeași pentru toate intervalele și ce bucată o știe numai subclasa?

Partea 3 — provocări

7. Clasa din mijloc care rămâne și ea abstractă

provocare

`OraCurs` și `Examen` au amândouă o sală; `Pauza` și `ActivitateLibera` nu. Bagă între ele o clasă intermediară, care moștenește `Interval`, ține sala și scrie **doar o parte** din metodele abstracte moștenite — pe cealaltă o lasă nescrisă. `OraCurs` și `Examen` o moștenesc pe ea.

Prima dată încearcă intenționat **fără** cuvântul `abstract` pe clasa din mijloc. Notează mesajul.

Gata când: orarul afișează la fel ca înainte, iar tu poți spune în scris câte clase din ierarhie sunt abstracte și de ce fiecare.

O clasă e obligată să fie abstractă atunci când rămâne cu datorii nerezolvate. Câte datorii are clasa din mijloc după ce scrie o parte din metode?

8. Starea ține de clasa abstractă, nu de subclase

provocare

Fă în așa fel încât ora de început și durata să existe **numai** în `Interval`, cu câmpuri `private` și constructor `protected`. Nicio subclasă nu mai declară câmpurile astea și niciuna nu le mai atinge direct.

Gata când: `grep -n "oraInceput\|durataMin"` pe fiecare subclasă dă `0`, iar programul afișează exact ce afișa înainte.

O clasă abstractă nu e doar o listă de metode nescrise — are voie și cu câmpuri, și cu constructor. Ce câștigi dacă ora există într-un singur loc și ce pierzi dacă fiecare subclasă își ține copia ei?

9. Aceeași operație, pe cealaltă ierarhie

provocare

Fă `Figura` abstractă și declară abstracte cele trei metode pe care acum le are cu corp gol sau cu `return null`. Compilează și lasă compilatorul să-ți spună, clasă cu clasă, ce ai uitat.

Apoi adaugă un `Triunghi` (trei puncte) și pune-l într-un `Desen` împreună cu celelalte figuri. Mută tot desenul și afișează-l.

Gata când: `Desen` nu conține nici `instanceof`, nici `cast`, nici `if` după tip; iar `Triunghi` se mută și se afișează corect fără nicio modificare în `Desen`.

Compară cu ce se întâmpla înainte: un `Triunghi` scris pe jumătate compila liniștit și afișa un rând gol. Acum, ce te oprește să-l lași pe jumătate?